

Tự động phân biệt với TensorFlow GradientTape

- Thực tế, Framework hiện đại như TensorFlow tự động hóa toàn bộ việc Lan truyền ngược bằng **Automatic Differentiation**.

```
1 import tensorflow as tf
2
3 x = tf.zeros(shape=()) # Gia tri khoi tao
4 with tf.GradientTape() as tape:
5     # Tape (Bang ghi) se luu lai do thi cac phep toan
6     y = 2 * x + 3
7
8 # Yeu cau Tape tinh gradient cua y doi voi x (dao ham)
9 grad_of_y_wrt_x = tape.gradient(y, x)
10
```

Thực hành (Từ đầu): NaiveDense

- Hãy xây dựng thủ công một lớp Dense sử dụng tensor ops.

```
1 import keras
2 from keras import ops
3
4 class NaiveDense:
5     def __init__(self, input_size, output_size, activation=None):
6         self.activation = activation
7         self.W = keras.Variable(shape=(input_size, output_size),
8                                   initializer="uniform")
9         self.b = keras.Variable(shape=(output_size,), initializer="zeros")
10
11     def __call__(self, inputs): # Thực hiện Forward Pass
12         x = ops.matmul(inputs, self.W) + self.b
13         if self.activation:
14             x = self.activation(x)
15         return x
16
```

Thực hành (Từ đầu): Vòng lặp Huấn luyện

- Khái quát lại quá trình `model.fit()` bằng đoạn code sau:

```
1 def one_training_step(model, images_batch, labels_batch):
2     with tf.GradientTape() as tape:
3         # 1. Forward pass để dự đoán và tính Loss
4         predictions = model(images_batch)
5         loss = ops.sparse_categorical_crossentropy(labels_batch, predictions)
6
7         average_loss = ops.mean(loss)
8
9         # 2. Backward pass (Tính gradient tự động nhờ Tape)
10        gradients = tape.gradient(average_loss, model.weights)
11
12        # 3. Cập nhật trọng số bằng Optimizer (như SGD)
13        update_weights(gradients, model.weights)
14
15    return average_loss
```

Tổng kết Chương 2

- **Tensors:** Nền tảng cấu trúc dữ liệu của học sâu. Bao gồm các bậc 0D, 1D, 2D, 3D, 4D v.v.
- **Phép toán Tensor:** Phép nhân ma trận, phép cộng, ReLU... là những biến đổi hình học tinh vi giúp gỡ rối đa diện dữ liệu.
- **Trọng số (Weights):** Tham số học hỏi chứa toàn bộ "kiến thức" của mô hình mạng.
- **Loss function & Optimizers:** Loss đo lường độ chính xác (mục tiêu cần tối thiểu hóa). Optimizer như SGD giảm Loss thông qua đi ngược Gradient.
- **Backpropagation:** Sử dụng Đồ thị tính toán và Quy tắc chuỗi để tính đạo hàm ngược một cách nhanh chóng qua cơ chế phân biệt tự động (GradientTape).